

NexaFlow

Predicción de demanda

Maqueta del informe de la EPE 3

Teoría y Aplicaciones de la Inteligencia Artificial

IPCHILE

Estudiante: [Completar]

Carrera y sección: [Completar]

Docente: [Completar]

Fecha de entrega: [Completar]

Versión de trabajo con evidencia técnica local

Datos sintéticos

Validación empresarial y usabilidad pendientes

Cómo usar esta maqueta

Desarrolla los apartados marcados Completar, conserva la trazabilidad de los resultados y reemplaza las cifras si vuelves a entrenar. Esta versión organiza el trabajo y no constituye un informe final listo para entregar.

Introducción

Este documento organiza el desarrollo de la EPE 3 mediante una aplicación de predicción de demanda para el caso NexaFlow. La propuesta transforma las variables operativas de una zona en una estimación diaria de unidades y permite consultar el modelo desde una interfaz web conectada a una API.

El prototipo es ejecutable, pero los resultados técnicos provienen de datos sintéticos. No se dispone de registros empresariales, de la EPE 2 completa ni de pruebas con usuarios. Por eso, las métricas no acreditan mejoras reales de inventario, ahorro ni capacidad productiva.

La pauta solicita informe en PDF, código en GitHub y presentación PowerPoint con evidencia. El informe utiliza Calibri 12, márgenes de 3 cm, interlineado 1,5 y alineación justificada. El desarrollo cubre diseño, implementación, evaluación y documentación, con evidencia visual de la ejecución local.

El alcance implementado comprende demanda agregada por zona, comparación de modelos, servicio FastAPI, formulario Flask, pruebas automatizadas y carga HTTP local. La evaluación empresarial y la usabilidad con participantes se deben completar por separado.

1 Diseño del problema empresarial

Parte 1 de la pauta. NexaFlow se utiliza como caso de continuidad sugerido por el estudiante. La necesidad propuesta es anticipar la demanda diaria por zona para apoyar la revisión de inventario. Debe verificarse que esta necesidad coincide con el diagnóstico y los antecedentes de la evaluación anterior.

La unidad de análisis implementada es una zona en un día. El objetivo es estimar unidades agregadas; no existe una variable de producto ni una optimización de rutas. Para desagregar por producto se necesitarían su identificador, historial suficiente y un rediseño de la preparación y evaluación de datos.

El usuario previsto es una persona responsable de planificación o reposición. Ingresar zona, día, mes, promoción y demanda de siete días antes. La aplicación presenta la estimación y la advertencia de datos sintéticos. La decisión final requiere revisar existencias, pedidos pendientes, capacidad y restricciones operativas.

El beneficio esperado es reducir incertidumbre al planificar. Esta expectativa debe comprobarse mediante un piloto que compare decisiones y costos con y sin apoyo del modelo. No se puede deducir ahorro financiero únicamente desde MAE o R^2 .

Trazabilidad propuesta con EPE 2: diagnóstico a verificar = dificultad para anticipar demanda por zona; causa raíz hipotética = planificación basada en demanda pasada sin integrar calendario y promoción; objetivo empresarial = revisar inventario y despacho antes de la ruta. La implementación traduce ese diagnóstico en variables, modelo y consulta web. Revisar el texto exacto de EPE 2 antes de entregar.

2 Objetivos y requisitos de la solución

Objetivo general propuesto: desarrollar y evaluar un prototipo web que estime demanda diaria por zona y comunique el resultado mediante una API. Los objetivos específicos son construir un conjunto reproducible, comparar modelos con separación temporal y comprobar el funcionamiento del flujo de consulta.

Requisitos funcionales implementados: ingresar cinco variables, solicitar una predicción, mostrar unidades, rechazar entradas fuera de rango y comunicar indisponibilidad del servicio. La API expone POST /predict y GET /health; la documentación interactiva se encuentra en /docs.

Requisitos de calidad propuestos: conservar entradas cuando ocurre un error, mostrar etiquetas comprensibles, responder sin exponer detalles internos y mantener resultados reproducibles. La carga local sirve como observación inicial de tiempos, no como garantía de servicio.

Criterio de aceptación del modelo: comparar el error con una línea base de persistencia semanal y evaluar si la magnitud resulta útil para el negocio. La reducción de error necesaria debe acordarse con responsables operativos, considerando costos de sobrestock y desabastecimiento.

Completar: definir usuarios, volumen de consultas esperado, objetivo de latencia y umbral de error aceptable. Marcar cada requisito como implementado, verificado o pendiente y vincularlo con una evidencia concreta.

3 Datos y preparación temporal

El generador produce 2896 filas útiles de cuatro zonas entre 2023 y 2024, con semilla 42. Simula una demanda aditiva influida por zona, fin de semana, promoción, estacionalidad mensual y ruido. Esta formulación facilita una demostración reproducible y puede favorecer la regresión lineal.

Las variables son zona (1 a 4), día de semana (0 a 6), mes (1 a 12), promoción (0 o 1) y `demand_lag7`. El objetivo demanda es una cantidad continua de unidades. La fecha determina la separación temporal y no entra directamente en el estimador.

El rezago se construye dentro de cada zona con el valor observado siete días antes. Se eliminan los primeros siete registros de cada zona. El calendario sintético es completo; con registros reales habría que distinguir días sin actividad de datos faltantes y reindexar antes de calcular el rezago.

La codificación one-hot de zona, día y mes se encuentra dentro del pipeline. Cada entrenamiento aprende sus transformaciones utilizando solo su partición de entrenamiento. El conjunto de test no interviene en la selección del modelo.

Con datos reales, el ERP aportaría fecha, zona/ruta, productos y unidades de pedidos o ventas; el WMS aportaría inventario disponible, quiebres y movimientos; el calendario comercial aportaría promociones. Se debería construir `demand_lag7` por ruta y distinguir ventas censuradas por falta de stock de demanda verdadera. Son campos propuestos, no datos actualmente integrados.

4 Arquitectura y flujo de predicción

La arquitectura separa la presentación de la inferencia. El navegador envía el formulario a Flask en el puerto 5000. Flask transforma los campos numéricos y realiza una solicitud HTTP a FastAPI en el puerto 8000. FastAPI valida los valores y consulta el modelo cargado en memoria.

El entrenamiento ocurre fuera de las solicitudes web. `train_model.py` guarda el pipeline, nombres de variables y nombre del modelo en `artifacts/modelo.pkl`. La API carga ese archivo al iniciar; si no existe, devuelve un estado de indisponibilidad con una instrucción de recuperación.

El resultado incluye `prediccion_demanda`, `unidad`, `modelo` y origen sintético de los datos. Flask muestra la cantidad y mantiene los campos seleccionados. Un timeout evita que la web espere indefinidamente cuando falla la comunicación.

Los archivos se ubican con rutas relativas al código cuando corresponde. El endpoint del servicio puede configurarse con `NEXAFLOW_API_URL`. El prototipo se ejecuta en la interfaz local del equipo y requiere iniciar ambos procesos.

Completar: incorporar un diagrama propio del flujo navegador, Flask, FastAPI y modelo. Explicar los puertos, la respuesta ante fallos y por qué la separación permite cambiar la interfaz sin volver a entrenar.

5 Entrenamiento y selección del modelo

Parte 2 de la pauta. Se comparan una regresión lineal y cuatro configuraciones de Random Forest con 100 árboles, profundidad máxima de 6 o 12 y mínimo de 2 o 5 muestras por hoja. Esta búsqueda acotada permite mostrar ajuste de hiperparámetros sin afirmar que se exploraron todas las alternativas.

El primer 80% de las fechas aporta 2316 registros de desarrollo. Sobre estas fechas se ejecuta TimeSeriesSplit con cinco ventanas crecientes. Las cuatro zonas de una fecha siempre pertenecen a la misma partición. El test final contiene 580 registros desde 2024-08-09.

La selección utiliza el menor MAE medio de validación. Después se ajusta el candidato seleccionado con todo el bloque de desarrollo y se evalúa una vez en el test reservado. Se conserva ese modelo evaluado para servir predicciones, sin reentrenarlo con el test.

El candidato seleccionado fue Lineal, con MAE medio de validación 9.26. Los resultados detallados por ventana y los límites de fechas están en metricas.json. La comparación no demuestra que ese algoritmo será superior con demanda real.

Completar: insertar tabla con los cinco candidatos, comentar estabilidad entre ventanas y relacionar la elección con el análisis previo. Explicar que el rezago del test usa observaciones pasadas disponibles para cada predicción diaria, no un pronóstico multihorizonte.

Variable transformada con mayor magnitud en el modelo: remainder__promocion (coeficiente 22.14). Coeficientes del modelo lineal; describen asociación condicional, no causalidad. Categorías one-hot y escalas distintas limitan comparaciones directas.

6 Interfaz y contrato de la API

La interfaz permite seleccionar ruta, fecha mediante calendario, promoción y demanda observada hace siete días. Deriva día y mes de la fecha, informa el tiempo de espera y muestra una recomendación según el promedio sintético reciente de la ruta. Expone las métricas, el historial y el error RMSE de test; el $\pm$ RMSE es una referencia aproximada, no un intervalo de confianza.

El contrato recibe enteros estrictos para zona, día, mes y promoción. La demanda debe ser numérica, finita y no negativa, con un límite técnico de 100000. Se rechazan campos adicionales. El límite requiere revisión al incorporar el contexto empresarial.

Una solicitud válida responde con código 200. FastAPI devuelve 422 ante un cuerpo inválido y 503 si el modelo no está cargado. La web presenta mensajes comprensibles y devuelve 400 por datos incorrectos o 503 por indisponibilidad. La documentación /docs permite probar el contrato.

En la ejecución comprobada, zona 1, sábado, diciembre, promoción activa y demanda_lag7 de 95.0 produjeron 127.8 unidades. El request y response completos se guardan en ejemplo_api.json. Es una predicción de demostración sobre el modelo sintético.

La validación de la API permanece necesaria aunque exista un formulario: clientes externos pueden omitir la interfaz. La versión final debe incluir capturas actualizadas del formulario, resultado, mensaje de error y /docs.

7 Métricas y análisis de errores

Parte 3 de la pauta. Se utilizan MAE y RMSE porque el objetivo es continuo. MAE mide el error absoluto promedio en unidades; RMSE da mayor peso a errores grandes. R^2 es una medida complementaria de variación explicada. Precision, Recall y F1 corresponden a clasificación y no se aplican directamente a esta regresión.

En el test reservado, el modelo obtuvo MAE 6.41 unidades, RMSE 8.07 unidades y R^2 0.842. La persistencia semanal, que repite demanda_lag7, obtuvo MAE 13.70 y RMSE 17.51. Los valores se calcularon sobre las mismas filas.

El menor error frente a la línea base respalda la demostración. No prueba reducción de quiebres de stock ni ahorros reales. El $\pm$ RMSE expuesto en la web resume el error cuadrático de test y no garantiza cobertura para una predicción individual.

El archivo predicciones_test.csv permite revisar errores por zona, promociones y fecha. Ese análisis debe acompañar las métricas globales para detectar segmentos donde una buena media oculte un comportamiento deficiente.

El gráfico de residuos compara predicción y error real menos predicción, coloreado por zona. Su dispersión en torno a cero sirve como inspección visual; por sí sola no demuestra ausencia de sesgo sistemático. El análisis por zona, promoción y tipo de día está en metrics.json.

Método	MAE	RMSE	R^2
Modelo seleccionado	6.41	8.07	0.842
Persistencia semanal	13.70	17.51	0.254

Tabla 1. Comparación sobre las mismas 580 filas de test; MAE y RMSE en unidades.

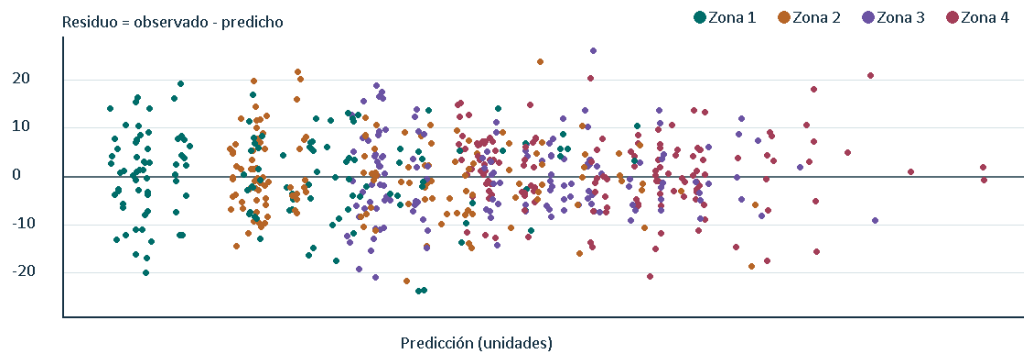


Figura 1. Residuos por zona en el test temporal; fuente: predicciones_test.csv.

8 Pruebas funcionales y rendimiento

La suite automatizada terminó con 22 pruebas aprobadas. Incluye predicción válida, validación de rangos y tipos, claves adicionales, cuerpos incompletos, modelo ausente, carga del formulario, comunicación web/API, caída de la API, estado de salud del flujo completo, documentación sin CDN, información visible del modelo y límites temporales.

La integración de prueba utiliza TestClient para FastAPI y el cliente de Flask. Además se ejecutó el flujo real por HTTP y se capturó su resultado desde un navegador. Las pruebas no equivalen a una auditoría de seguridad ni a una evaluación con usuarios.

La medición local envió 100 solicitudes con concurrencia de 10 a cada flujo. API: 0 errores y p95 41.9 ms. Web con consulta a API: 0 errores y p95 85.5 ms. Se hizo una solicitud previa de calentamiento por flujo.

Los resultados corresponden a una ejecución breve en Windows con servicios locales y sin tráfico externo. No describen capacidad sostenida, latencia de internet ni tolerancia a fallos bajo condiciones productivas. El script incluye timeout y registra solicitudes fallidas.

Completar: repetir benchmark en el equipo de presentación, registrar hardware, versiones y fecha. Aumentar escenarios de carga según el uso esperado y explicar qué se optimizó tras observar los resultados.

9 Usabilidad y riesgos operativos

La inspección del navegador verificó que el formulario permite solicitar una predicción y que la página móvil no presenta desbordamiento horizontal. Esto es una comprobación técnica de interfaz. No se realizaron sesiones con participantes y no se dispone de resultados de usabilidad humana.

Protocolo de cinco minutos: pedir a tres a cinco personas que obtengan una predicción para la ruta 2 el próximo sábado con promoción activa. Cronometrar desde la instrucción hasta que identifican la cifra y registrar si completaron sin ayuda y un comentario. El archivo PROTOCOLO_USABILIDAD.md contiene la hoja en blanco; no hay resultados inventados.

Se proponen como metas iniciales al menos 80% de tareas completas y mediana inferior a 60 segundos. Estas metas deben acordarse antes de las sesiones; no son resultados obtenidos. PLAN_DE_TRABAJO.md contiene una tabla para registrar las observaciones.

Riesgos: datos sintéticos poco representativos, cambio de patrón de demanda, rezagos incorrectos y uso del resultado como orden automática. El prototipo no incluye autenticación ni despliegue productivo. Solo carga un modelo generado localmente y necesita validación adicional antes de uso operacional.

Completar: aplicar el protocolo, presentar resultados anónimos y describir al menos una mejora basada en observación. Definir quién revisará deriva de datos, frecuencia de reentrenamiento y condiciones para retirar un modelo.

10 Documentación y presentación de resultados

Parte 4 de la pauta. El paquete contiene código, dependencias fijadas, conjunto sintético, modelo entrenado, métricas, pruebas, carga y capturas. README.md explica cómo crear el entorno, entrenar, iniciar los servicios y consultar la API. El modelo conserva el hash registrado en metrics.json para identificar el archivo evaluado.

La presentación ejecutiva resume problema, solución, método, resultados medidos y límites. Incluye una captura real de la aplicación y cifras de la ejecución local. Antes de entregar, debe incorporar nombre del estudiante, identificación del curso y enlace al repositorio publicado.

Repositorio del proyecto: <https://github.com/RenatoLV/EPE3-IPCHILE-ING>. Antes de entregar, comprobar acceso del docente y reproducir desde un clon limpio. El informe y la presentación deben corresponder al mismo commit publicado.

Guion sugerido: explicar la decisión empresarial, demostrar una predicción, describir la separación temporal, presentar MAE/RMSE y mencionar las limitaciones. Cerrar con los pasos necesarios para validar datos y utilidad empresarial. Evitar afirmar que el sistema está listo para producción.

Completar: URL real del repositorio, inventario de evidencias, instrucciones de reproducción verificadas y captura de la versión entregada. Revisar que el desarrollo final tenga de diez a quince páginas sustantivas, sin contar portada, introducción, conclusión ni bibliografía.

11 Evidencia visual del flujo web

Captura de la versión final en ejecución local con Docker Compose. El ejemplo selecciona la ruta 1, el sábado 19 de septiembre de 2026, promoción activa y demanda observada de 95 unidades siete días antes. La respuesta fue 113,7 unidades con referencia de $\pm 8,1$ unidades de RMSE de test.

La recomendación compara la estimación con el promedio de los últimos 21 registros sintéticos de esa ruta. Este promedio sirve como contexto de demostración; el historial termina en 2024 y no representa la demanda real de 2026.

Preparar una estimación

Completa los datos que se conocen antes de organizar la ruta.

Ruta de reparto referencial

Ruta 1 · Centro y Guayacán

Centro y Guayacán

Centro, Guayacán y El Llano. Ruta referencial de cobertura urbana central.

Fecha que se desea planificar

El calendario calcula automáticamente el día de semana y el mes.

19-09-2026

sábado, 19 de septiembre de 2026

¿Habrá promoción activa?

No

Sí

Demanda observada hace 7 días

Usa las unidades reales de la misma ruta, para el mismo día de la semana anterior.

95

Estimar demanda de la ruta

Resultado para la ruta

✓ Estimación calculada

113.7

unidades estimadas para sábado 19 de septiembre de 2026

113.7 ± 8.1 unidades

Referencia: RMSE medido en test. Es un error típico aproximado, no un intervalo de confianza ni una garantía para esta ruta.

Siguiente revisión recomendada

Demanda sobre el promedio reciente de la ruta: revisa stock y capacidad de despacho antes de confirmar.

Promedio de los últimos 21 días sintéticos registrados para esta ruta: 97.3 unidades.

Ruta seleccionada

Centro y Guayacán

Sector referencial

Centro, Guayacán y El Llano

Uso responsable

La predicción apoya una decisión humana y debe validarse con registros reales.

Sobre este ejemplo

Las rutas son referenciales para contextualizar Coquimbo. El modelo fue entrenado con datos sintéticos y no representa la demanda real de los sectores nombrados.

Figura 2. Formulario y resultado del prototipo en ejecución local.

NexaFlow | Maqueta EPE 3 | 14

12 Evidencia de API y despliegue

La documentación /docs es servida por FastAPI y funciona sin recursos externos. Incluye el contrato de POST /predict, un formulario de prueba y acceso al esquema OpenAPI generado en /openapi.json. El resultado de la prueba queda visible en la captura.

Docker Compose inició dos contenedores con estados healthy: api y web. El healthcheck de web consulta /health de la API con un timeout breve, por lo que refleja la disponibilidad del flujo completo. Este estado es local y no equivale a monitoreo productivo.

NexaFlow API
Documentación Interactiva - FastAPI - prototipo con datos sintéticos

La interfaz Flask consume esta API para estimar demanda diaria de una ruta. El modelo y las métricas son de demostración; no representan ventas reales de Coquimbó. El esquema OpenAPI generado por FastAPI está en </openapi.json>.

GET /health

Comprueba que el modelo está cargado. Devuelve 200 si está disponible o 503 si falta.
[Consultar estado ahora](#)

POST /predict

Recibe cinco variables y devuelve la demanda estimada en unidades por día. Los campos adicionales o fuera de rango producen 422.

Zona (1-4)

Día de semana (0=Jueves, 6=domingo)

Mes (1-12)

Promoción (0=no, 1=sí)

Demanda observada hace 7 días

[Enviar solicitud de prueba](#)

HTTP 200 - Solicitud correcta

```
{
  "prediccion_demanda": 127.8,
  "unidad": "unidades/día",
  "modelo": "lineal",
  "datos": "sintéticos"
}
```

Ejemplo de solicitud

```
{
  "zona": 1,
  "dia_semana": 5,
  "mes": 12,
  "promocion": 1,
  "demanda_lag": 95.0
}
```

La API devuelve prediccion_demanda, unidad, modelo y datos. Consulta el esquema OpenAPI para tipos y validaciones completas.

Figura 3. Prueba interactiva de /docs en FastAPI.

Conclusión

El prototipo demuestra una integración funcional entre una interfaz Flask, una API FastAPI y un modelo de Scikit-learn. La separación temporal, las pruebas automatizadas y la medición HTTP ofrecen evidencia reproducible del funcionamiento técnico en el entorno local.

La regresión lineal seleccionada alcanzó MAE 6.41 y RMSE 8.07 en el test sintético. Superó a la persistencia semanal en ese experimento. La estructura aditiva del generador y la ausencia de datos reales limitan cualquier extrapolación a la empresa.

Para completar la actividad se debe verificar el vínculo con la evaluación anterior, incorporar antecedentes empresariales verificables y realizar las pruebas de usabilidad. La validación con demanda real permitirá discutir si los errores son aceptables para la decisión de inventario.

Completar: redactar una reflexión propia sobre lo aprendido, decisiones de diseño, dificultades y mejoras. Distinguir resultados alcanzados, expectativas y trabajo futuro.

Bibliografía

IPCHILE. (2024). Diseño, implementación y evaluación de una aplicación web con Machine Learning en un entorno empresarial real [Pauta de actividad evaluativa]. Material proporcionado por el estudiante.

Scikit-learn developers. (s. f.). Cross-validation: Evaluating estimator performance. Scikit-learn. Recuperado el 14 de septiembre de 2026, de https://scikit-learn.org/stable/modules/cross_validation.html

FastAPI. (s. f.). Request body. Recuperado el 14 de septiembre de 2026, de <https://fastapi.tiangolo.com/tutorial/body/>

Pallets. (s. f.). Quickstart. Flask documentation. Recuperado el 14 de septiembre de 2026, de <https://flask.palletsprojects.com/en/stable/quickstart/>

Completar: agregar la referencia exacta de la EPE 2 y del material docente que efectivamente se utilice. Verificar autor institucional y año de la pauta contra la información del curso. No citar apuntes no revisados como respaldo de conclusiones.